# Time and Space Complexity Analysis

## 1. Time Complexity

- Time Complexity represents how the execution time of an algorithm grows with respect to input size (`n`).
- It is measured by counting the number of basic operations, not actual execution time.
- Expressed using **Asymptotic Notations**:
  - `O()` – Upper bound (commonly used for worst-case analysis).
  - `Ω()` – Lower bound.
  - `Θ()` – Tight bound.

## 2. Space Complexity

- Space Complexity represents the amount of memory required by an algorithm with respect to input size (`n`).
- It includes:
  - Input space.
  - Auxiliary space (extra memory used by the algorithm).
  - Recursion stack space.
- In algorithm analysis, auxiliary space is commonly considered separately from input storage.

## 3. How to Calculate Time Complexity?

- **Simple Statement:** O(1)
  - Constant number of operations.
- **Single Loop:** O(n)
  - Loop executes `n` times.
- **Nested Loops:** $O(n^2)$
  - Two nested loops, each executing `n` times.

- **Three Nested Loops:** $O(n^3)$
  - Three nested loops, each executing `n` times.
- **Loop with Increment by Constant:**
  - Example: `i = i + 2`
  - Complexity: O(n).
- **Loop with Multiplication/Division:**
  - Example: `i = i * 2`
  - Complexity: O(log n).
- **Consecutive Loops:** Add their complexities and retain the dominant term.
  - $O(n) + O(n^2) = O(n^2)$.
- **Nested Loops:** Multiply iteration counts.
  - $O(n) \times O(n) = O(n^2)$.

## 4. How to Calculate Space Complexity?

- Fixed number of variables: O(1).
- Array of size `n`: O(n).
- 2D array of size `n × n`: $O(n^2)$.
- Recursion with depth `n`: O(n) stack space.
- Recursion with depth `log n`: O(log n) stack space.
- Creating an additional array of size `n`: O(n) auxiliary space.

## 5. Common Time Complexities

| Complexity | Name | Example |
| --- | --- | --- |
| O(1) | Constant | Array element access |

| Complexity | Name | Example |
|---|---|---|
| O(log n) | Logarithmic | Binary Search |
| O(n) | Linear | Linear Search |
| O(n log n) | Linearithmic | Merge Sort |
| $O(n^2)$ | Quadratic | Bubble Sort |
| $O(n^3)$ | Cubic | Three nested loops |
| $O(2^n)$ | Exponential | Naive Fibonacci recursion |
| O(n!) | Factorial | Generating all permutations |

## 6. Important Rules for MCQs

- Ignore constant factors: O(2n) = O(n).
- Ignore lower-order terms: $O(n^2 + n + 1) = O(n^2)$.
- Sequential operations: Add complexities.
- Nested loops: Multiply iteration counts.
- Always consider the number of iterations, not just the number of loops.
- For multiple input variables, retain all relevant variables: O(n + m).
- Worst-case, average-case and best-case complexities may differ.
- Recursive algorithms must include recursion stack space in space analysis.
- Two separate loops do not necessarily mean $O(n^2)$.
- A loop that halves the input in every iteration generally has O(log n) complexity.